BÁO CÁO THỰC HÀNH OOP - TUẦN 7

Chủ đề: Đa Hình Và Generic Trong Quản Lý Thiết Bị Giảng Dạy

MSSV: 202418834 Họ tên: Trần Tuấn Anh

Ngày 23 tháng 10 năm 2026

1 Link Github - Mã nguồn

Kho lưu trữ mã nguồn trên Github: https://github.com/tuananhtran109/OOP-Ex07

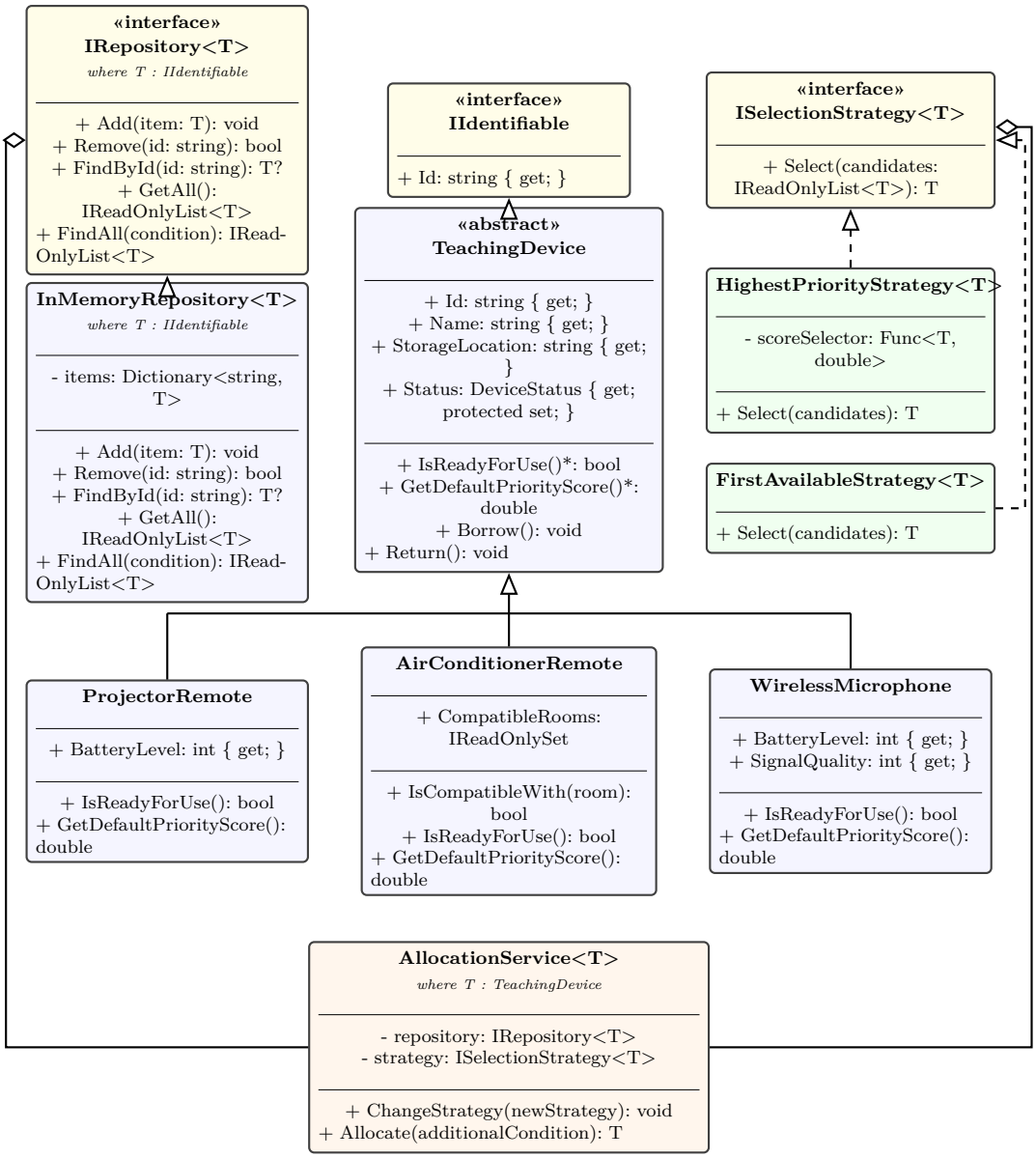
2 Phân tích yêu cầu nghiệp vụ

Hệ thống quản lý và cấp phát thiết bị phục vụ giảng dạy tại trường đại học được xây dựng nhằm giải quyết bài toán quản trị danh mục thiết bị, kiểm tra khả năng sẵn sàng và quy trình cấp phát tối ưu:

- **Quản lý danh mục thiết bị giảng dạy:** Quản lý các thiết bị phục vụ giảng đường tiêu biểu gồm *Điều khiển máy chiếu*, *Điều khiển điều hòa*, và *Micro không dây*. Mỗi thiết bị có các thuộc tính cơ sở: Mã thiết bị (`Id`), Tên thiết bị (`Name`), Vị trí lưu trữ trong kho (`StorageLocation`) và Trạng thái hoạt động (`Status`: `Available`, `Borrowed`, `UnderMaintenance`).
- **Khả năng sẵn sàng và Điểm ưu tiên cấp phát (Tính Đa hình):** Các loại thiết bị khác nhau có cách kiểm tra khả năng sẵn sàng (`IsReadyForUse()`) và thuật toán tính điểm ưu tiên (`GetDefaultPriorityScore()`) đặc thù:
  - *Điều khiển máy chiếu:* Quản lý mức pin (0–100%). Chỉ sẵn sàng khi trạng thái là `Available` và pin từ 20% trở lên. Điểm ưu tiên mặc định bằng chính mức pin.
  - *Điều khiển điều hòa:* Quản lý tập phòng tương thích (`CompatibleRooms`). Sẵn sàng khi trạng thái là `Available`. Khi cấp phát cho một phòng cụ thể, điều khiển phải tương thích với phòng đó (`IsCompatibleWith(roomId)`). Điểm ưu tiên mặc định bằng 100 nếu tương thích, ngược lại bằng 0.
  - *Micro không dây:* Quản lý mức pin và chất lượng tín hiệu sóng truyền (đều từ 0–100%). Sẵn sàng khi `Available`, pin từ 30% trở lên và tín hiệu từ 40% trở lên. Điểm ưu tiên mặc định bằng trung bình cộng giữa pin và tín hiệu: (Pin + Tín hiệu)/2.
- **Kho lưu trữ tổng quát (Generic Repository):** Kho lưu trữ `IRepository<T>` và lớp cài đặt `InMemoryRepository<T>` được thiết kế tổng quát với ràng buộc kiểu `where T : IIdentifiable`. Kho hỗ trợ các thao tác CRUD và lọc theo điều kiện (`FindAll(Predicate<T>)`). Việc sử dụng `Dictionary<string, T>` giúp tra cứu `FindById` và kiểm tra trùng mã đạt hiệu năng tối ưu $O(1)$, đồng thời tái sử dụng mã nguồn cho mọi kiểu thiết bị mà không cần nhân bản.
- **Chiến lược lựa chọn thiết bị (Strategy Pattern):** Sử dụng giao diện `ISelectionStrategy<T>` để trừu tượng hóa thuật toán. Cài đặt đa hình nhiều chiến lược: `HighestPriorityStrategy<T>` (chọn theo điểm ưu tiên) và `FirstAvailableStrategy<T>` (chọn ứng viên đầu tiên). Cho phép hoán đổi chiến lược linh hoạt ngay trong thời gian chạy.
- **Dịch vụ cấp phát tổng quát (AllocationService):** Lớp `AllocationService<T>` (với `where T : TeachingDevice`) phối hợp giữa `IRepository<T>` và `ISelectionStrategy<T>`. Dịch vụ chỉ tương tác qua abstraction, tuân thủ nguyên lý Open/Closed (OCP) và Dependency Inversion (DIP), tuyệt đối không dùng rẽ nhánh kiểu (`if is`).
- **Bảo đảm tính nhất quán dữ liệu và Xử lý ngoại lệ:** Phát hiện sớm và xử lý các lỗi nghiệp vụ qua các ngoại lệ chuyên biệt (`DuplicateIdException`, `NoSuitableItemException`, `DeviceNotReadyException`). Trạng thái thiết bị chỉ chuyển sang `Borrowed` khi cấp phát thành công trọn vẹn.

3 Danh sách lớp và sơ đồ quan hệ

3.1 Sơ đồ lớp (Class Diagram)



Hình 1: Sơ đồ lớp (Class Diagram) của Hệ thống Quản lý Thiết bị Giảng dạy

3.2 Danh sách các lớp, giao diện và vai trò

Tên lớp / Interface	Phân loại	Mô tả vai trò và trách nhiệm
`DeviceStatus`	Enum	Định nghĩa 3 trạng thái hoạt động của thiết bị: `Available`, `Borrowed`, `UnderMaintenance`.
`IIdentifiable`	Interface	Interface định danh tổng quát với thuộc tính mã duy nhất `string Id { get; }`.
`TeachingDevice`	Abstract	Lớp cơ sở trừu tượng cho mọi thiết bị giảng dạy; quản lý Id, Name, StorageLocation, Status, `Borrow()`, `Return()` và 2 phương thức trừu tượng.
`ProjectorRemote`	Concrete	Đại diện điều khiển máy chiếu; quản lý dung lượng pin (0–100%), sẵn sàng khi `Available` và pin $\geq 20\%$.
`AirConditionerRemote`	Concrete	Đại diện điều khiển điều hòa; quản lý tập phòng tương thích và kiểm tra tương thích phòng học.
`WirelessMicrophone`	Concrete	Đại diện micro không dây; quản lý mức pin và chất lượng tín hiệu sóng truyền.
`IRepository<T>`	Generic Intf.	Interface kho lưu trữ tổng quát với CRUD và lọc điều kiện (`where T : IIdentifiable`).
`InMemoryRepository<T>`	Generic Cls.	Cài đặt kho trong bộ nhớ sử dụng `Dictionary<string, T>`, tối ưu tra cứu $O(1)$.
`ISelectionStrategy<T>`	Generic Intf.	Interface chiến lược lựa chọn thiết bị tối ưu từ danh sách ứng viên (Strategy Pattern).
`HighestPriorityStrategy<T>`	Generic Cls.	Chiến lược chọn đối tượng có điểm ưu tiên cao nhất với `Func<T, double>`.
`FirstAvailableStrategy<T>`	Generic Cls.	Chiến lược chọn ứng viên sẵn sàng đầu tiên trong danh sách.
`AllocationService<T>`	Generic Cls.	Dịch vụ cấp phát tổng quát (`where T : TeachingDevice`), phối hợp kho lưu trữ và chiến lược lựa chọn.
`DeviceException`	Exception	Ngoại lệ cơ sở cho toàn bộ các ngoại lệ nghiệp vụ quản lý thiết bị.

3.3 Hệ phân cấp các ngoại lệ dẫn xuất bắt buộc

- **`DuplicateIdException`:** Phát sinh khi thêm vào kho lưu trữ một đối tượng có mã định danh `Id` đã tồn tại. Chứa thuộc tính `DuplicateId`.
- **`NoSuitableItemException`:** Phát sinh khi không tìm thấy thiết bị nào trong kho sẵn sàng hoặc thỏa mãn điều kiện yêu cầu để cấp phát.
- **`DeviceNotReadyException`:** Phát sinh khi cố gắng mượn một thiết bị chưa đủ điều kiện sẵn sàng (`IsReadyForUse()` trả về false). Chứa thuộc tính `DeviceId` và `CurrentStatus`.
- **`InvalidOperationException`:** Phát sinh khi thao tác chuyển đổi trạng thái không hợp lệ (ví dụ: gọi `Return()` trên thiết bị không ở trạng thái `Borrowed`).

3.4 Thiết kế trước khi lập trình và trả lời câu hỏi định hướng

1. *Hành vi chung và thay đổi:* Các hành vi `Borrow`, `Return` quản lý trạng thái đặt tại lớp cơ sở `TeachingDevice`; các hành vi `IsReadyForUse`, `GetDefaultPriorityScore` được ghi đè đa hình tại các lớp con.
2. *Kho lưu trữ và Ràng buộc Generic:* Ràng buộc `where T : IIdentifiable` bảo đảm mọi đối tượng đều có `Id`. Nếu bỏ ràng buộc, `FindById` không thể truy cập `item.Id` ở compile-time mà phải dùng ép kiểu hoặc reflection gây mất an toàn kiểu.
3. *Chiến lược lựa chọn thiết bị:* Áp dụng mẫu Strategy (`ISelectionStrategy<T>`) thay cho chuỗi `if/else`, giúp dịch vụ cấp phát độc lập giải thuật và dễ dàng mở rộng chiến lược mới mà không sửa mã nguồn (nguyên lý OCP).
4. *Quyết định thiết kế và Trade-off:* Dùng `Dictionary<string, T>` thay vì `List<T>`. Đánh đổi thêm một ít bộ nhớ cho bảng băm nhưng đổi lại tốc độ tìm kiếm `FindById` và kiểm tra trùng mã đạt $O(1)$ vượt trội so với $O(n)$ của `List`.
5. *Tránh nhân bản mã và lạm dụng Generic:* Không tạo các lớp repository riêng biệt vì cấu trúc lưu trữ giống nhau; dùng `IRepository<T>` để tái sử dụng mã. Đồng thời không generic hóa mọi lớp trong hệ thống để tránh làm phức tạp thiết kế.

3.5 Phân tích mối quan hệ giữa các lớp

1. **Kế thừa đa hình:** `ProjectorRemote`, `AirConditionerRemote`, `WirelessMicrophone` kế thừa lớp trừu tượng `TeachingDevice`.
2. **Thực thi Giao diện:** `TeachingDevice` triển khai `IIdentifiable`; `InMemoryRepository<T>` triển khai `IRepository<T>`.
3. **Ràng buộc Generic:** Ràng buộc `where T : IIdentifiable` cho kho lưu trữ và `where T : TeachingDevice` cho dịch vụ cấp phát.
4. **Tập hợp & DI:** `AllocationService<T>` phụ thuộc vào các giao diện trừu tượng `IRepository<T>` và `ISelectionStrategy<T>`.

4 Kết quả chạy và kiểm thử hệ thống

Chương trình khởi tạo bộ dữ liệu mẫu gồm 3 ĐK máy chiếu (`PR01`–`PR03`), 2 ĐK điều hòa (`AC01`–`AC02`) và 3 micro không dây (`MC01`–`MC03`), sau đó tiến hành kiểm thử toàn diện qua 3 phân hệ chức năng:

4.1 Phần 1: Thao tác kho lưu trữ cơ bản (Mục 1–4)

Thực hiện các ca kiểm thử thao tác kho lưu trữ cơ bản:

- **1. Thêm thiết bị vào repository:** Nạp thành công 3 điều khiển máy chiếu, 2 điều hòa và 3 micro vào các kho lưu trữ tương ứng.
- **2. Thử thêm hai thiết bị trùng mã:** Thêm thiết bị trùng mã `PR01` → Bắt chính xác ngoại lệ nghiệp vụ `DuplicateIdException`.
- **3. Tìm thiết bị theo mã:** Tra cứu `PR02` trả về đúng đối tượng, tra cứu `PR999` trả về null chính xác.
- **4. Lọc các thiết bị sẵn sàng:** Lọc chính xác các máy chiếu đủ điều kiện (PR02 pin 55%, PR03 pin 90%), loại bỏ PR01 (pin 15% $<$ 20%).



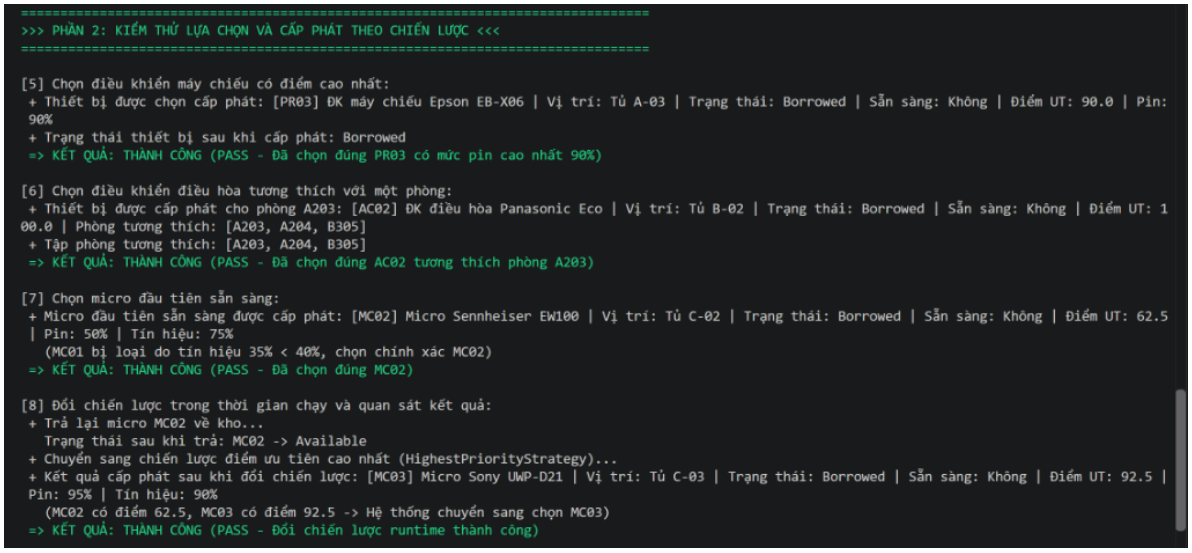
Hình 2: Kết quả chạy Phần 1: Thao tác kho lưu trữ cơ bản (Mục 1–4)

Nhận xét: Kho lưu trữ `InMemoryRepository<T>` đã nạp và quản lý thành công các loại thiết bị độc lập. Thao tác `FindById("PR02")` tìm đúng thiết bị, tìm mã `PR999` trả về null chính xác. Phương thức `FindAll(d => d.IsReadyForUse())` đã lọc chính xác các máy chiếu đủ điều kiện, loại bỏ các thiết bị chưa sẵn sàng đúng theo thiết kế.

4.2 Phần 2: Lựa chọn và cấp phát thiết bị theo chiến lược (Mục 5–8)

Thực hiện kịch bản lựa chọn tối ưu theo chiến lược và hoán đổi thuật toán lựa chọn tại runtime:

- **5. Chọn điều khiển máy chiếu có điểm cao nhất:** Dịch vụ dùng `HighestPriorityStrategy` chọn đúng `PR03` (pin 90% cao nhất) và chuyển sang trạng thái `Borrowed`.
- **6. Chọn điều khiển điều hòa tương thích với một phòng:** Cấp phát kèm điều kiện phòng học `device.IsCompatibleWith("A203")`, hệ thống chọn chính xác `AC02` (tương thích A203).
- **7. Chọn micro đầu tiên sẵn sàng:** Dịch vụ dùng `FirstAvailableStrategy` tự động bỏ qua `MC01` (tín hiệu 35% $<$ 40%) và chọn `MC02` là micro đầu tiên sẵn sàng.
- **8. Đổi chiến lược trong thời gian chạy và quan sát kết quả:** Hoàn trả `MC02` về kho (`Return()`); gọi `ChangeStrategy()` để chuyển sang `HighestPriorityStrategy`. Cấp phát lại, hệ thống chọn chính xác `MC03` (điểm 92.5) thay vì `MC02` (điểm 62.5).



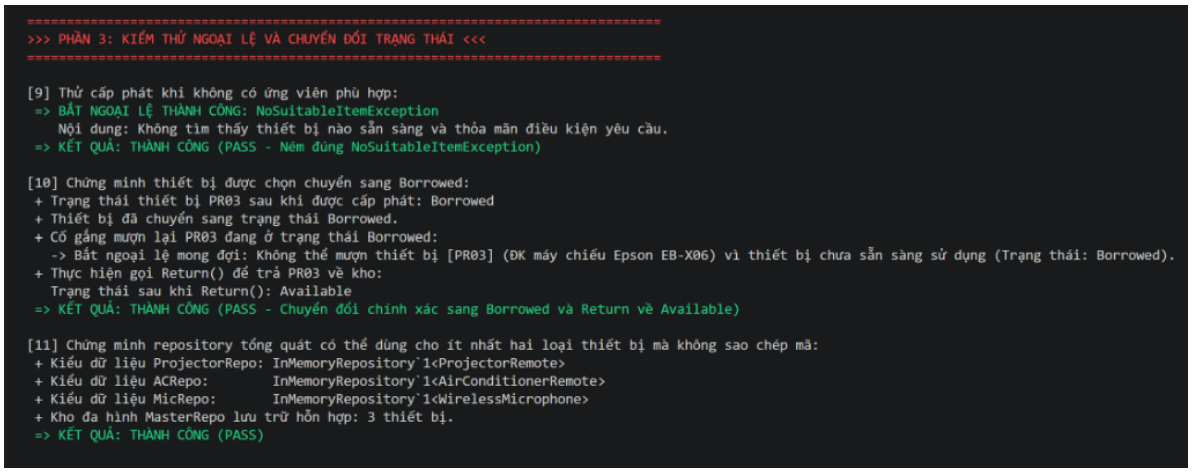
Hình 3: Kết quả chạy Phần 2: Lựa chọn và cấp phát thiết bị theo chiến lược (Mục 5–8)

Nhận xét: Dịch vụ cấp phát `AllocationService<T>` đã kết hợp hoàn hảo giữa `IRepository<T>` và `ISelectionStrategy<T>`. Việc hoán đổi chiến lược tại runtime diễn ra mượt mà thông qua phương thức `ChangeStrategy()`, giúp thay đổi hành vi lựa chọn từ phần tử đầu tiên sang phần tử có điểm ưu tiên cao nhất mà không cần khởi tạo lại dịch vụ.

4.3 Phần 3: Kiểm thử ngoại lệ, trạng thái và kho tổng quát (Mục 9–11)

Kiểm thử các tình huống biên, xử lý ngoại lệ nghiệp vụ và chứng minh tính tổng quát:

- **9. Thử cấp phát khi không có ứng viên phù hợp:** Yêu cầu cấp phát cho phòng `C999` không tồn tại → Hệ thống phát hiện không có ứng viên và ném ngoại lệ `NoSuitableItemException`.
- **10. Chứng minh thiết bị được chọn chuyển sang Borrowed:** Sau cấp phát thiết bị chuyển `Borrowed`; cố mượn tiếp bị chặn (`DeviceNotReadyException`); gọi `Return()` thành công đưa thiết bị về `Available`.
- **11. Chứng minh repository tổng quát có thể dùng cho ít nhất hai loại thiết bị mà không sao chép mã:** Kho `InMemoryRepository<T>` dùng an toàn kiểu cho từng loại thiết bị độc lập cũng như kho đa hình `InMemoryRepository<TeachingDevice>`.



Hình 4: Kết quả chạy Phần 3: Kiểm thử ngoại lệ, trạng thái và kho tổng quát (Mục 9–11)

Chi tiết các ca kiểm thử bất thường và xử lý ngoại lệ:

1. *Thêm phần tử trùng Id:* Ném `DuplicateIdException`. Kho lưu trữ giữ nguyên dữ liệu gốc, không bị ghi đè.
2. *Thiết bị không đủ điều kiện pin/sóng:* Bị loại khỏi danh sách ứng viên thông qua kiểm tra đa hình `IsReadyForUse()`.
3. *Cấp phát không tìm thấy ứng viên:* Ném `NoSuitableItemException` khi không có thiết bị nào thỏa mãn điều kiện mượn.
4. *Mượn thiết bị chưa sẵn sàng:* Ném `DeviceNotReadyException` khi thiết bị đang ở trạng thái `Borrowed` hoặc `UnderMaintenance`.
5. *Hoàn trả thiết bị không ở trạng thái Borrowed:* Ném `InvalidOperationException` bảo đảm quy trình mượn - trả chặt chẽ.

Kết luận: Hệ thống quản lý và cấp phát thiết bị giảng dạy đã hoàn thành toàn bộ các yêu cầu chức năng và kiểm thử thực nghiệm, thể hiện sự kết hợp nhịp nhàng giữa hai cơ chế cốt lõi trong lập trình hướng đối tượng nâng cao:

- **Generic:** Cung cấp tính an toàn kiểu tại thời gian biên dịch (compile-time type safety) và tái sử dụng thuật toán lưu trữ, cấp phát mà không cần nhân bản mã nguồn.
- **Đa hình:** Cung cấp khả năng thay đổi hành vi kiểm tra sẵn sàng, tính điểm ưu tiên và hoán đổi thuật toán lựa chọn linh hoạt tại thời gian chạy (runtime).